

Midterm Exam 2

(7:05 PM – 8:20 PM : 75 Minutes)

- This exam will account for either 15% or 30% of your overall grade depending on your relative performance in midterm exam 1 and midterm exam 2. The higher of the two scores will be worth 30% of your grade, and the lower one 15%.
- There are three (3) questions worth 75 points in total. Please answer all of them in the spaces provided.
- There are twenty-two (22) pages, including nine (9) blank pages and one (1) page of appendices. Please use the blank pages if you need additional space for your answers.
- The exam is **open slides** and **open notes** (including **scribe notes**). But **no books** and **no computers** are allowed.

Good Luck!

Question	Pages	Parts	Points	Score
1. “Probabilistic” Fibonacci Numbers	2 – 6	(a) – (c)	5 + 10 + 10 = 25	
2. Parallel Recursive Selection Sort	8 – 11	(a) – (b)	10 + 15 = 25	
3. Store-Retrieve Lockers	14 – 19	(a) – (d)	5 + 5 + 3 + 12 = 25	
Total			75	

NAME: _____

SBU ID: _____

```

PROB-FIB(  $n$  )
1. if  $n < 1$  then  $x \leftarrow 0$ 
2. elseif  $n = 1$  then  $x \leftarrow 1$ 
3. else
4.    $r \leftarrow \text{RANDOM}( )$            {choose a real number
                                         uniformly at random from  $(0, 1]$ }
5.   if  $r \leq 0.5$  then
6.      $x \leftarrow \text{PROB-FIB}( n - 1 )$ 
7.   else
8.      $x \leftarrow \text{PROB-FIB}( n - 2 )$ 
9.   endif
10. endif
11. return  $x$ 

```

Figure 1: When called with an integer $n \geq 0$ as a parameter, $\text{PROB-FIB}(n)$ will return what we will call the n -th “probabilistic” Fibonacci number.

Question 1. [25 Points] “Probabilistic” Fibonacci Numbers. The function given in Figure 1 computes what we will call “probabilistic” Fibonacci numbers. When supplied with an integer $n \geq 0$ as a parameter, it will return the n -th probabilistic Fibonacci number f_n . Clearly, $f_0 = 0$ and $f_1 = 1$, but for $n > 1$, f_n does not have a fixed value.

This question asks you to compute the expected running time of $\text{PROB-FIB}(n)$ for $n \geq 0$.

- (a) [5 Points] Let t_n be the expected running time of $\text{PROB-FIB}(n)$ for $n \geq 0$. We claim that t_n can be described by the following recurrence relation, where c_1 and c_2 are positive constants:

$$t_n \leq \begin{cases} c_1, & \text{if } n \leq 1, \\ \frac{1}{2}t_{n-1} + \frac{1}{2}t_{n-2} + c_2, & \text{otherwise,} \end{cases}$$

Justify this recurrence.

Solution

If $n = 1$, t_n will pass through two if statements before returning. If $n = 0$, there will only be one. Evaluating an if statement takes a small constant factor, and likewise, evaluating two takes a small constant factor (which is bigger than just one). Thus, $t_n \leq c_1$ if $n \leq 1$, where c_1 is the small constant time it takes to evaluate two if statements, then return.

2 points for justifying $t_n \leq c_1$ for $n \leq 1$

If $n > 1$, we pass through the if statements, create a random value, then come to an if/else. If the random value satisfies the if condition, then we will skip the rest of the if/else statement.

If it does not, we will need to pass over the else. This takes a very small amount of time, but still some constant factor. The original if/elseif/else, random generation, and doing another if/else are represented by c_2 , and if the if on line 5 is satisfied, c_2 is an overestimate.

1 point for justifying c_2

Either way, if $n > 1$, we either recurse on $\text{PROB-FIB}(n - 1)$ or $\text{PROB-FIB}(n - 2)$. These will have running times t_{n-1} and t_{n-2} , respectively. There's a $\frac{1}{2}$ chance we enter t_{n-2} , and $\frac{1}{2}$ chance we enter t_{n-1} . So we do $\frac{1}{2}t_{n-1} + \frac{1}{2}t_{n-2}$ as the expected running time.

2 points for justifying the $\frac{1}{2}t_{n-1} + \frac{1}{2}t_{n-2}$, and the $\leq$ part of this case.

All of this combines to the recurrence given.

- (b) [**10 Points**] Let us simplify the recurrence from part (a) to the following (by choosing $c_1 = c_2 = 1$ and replacing the ' $\leq$ ' with an '='.)

$$t_n = \begin{cases} 1, & \text{if } n \leq 1, \\ \frac{1}{2}t_{n-1} + \frac{1}{2}t_{n-2} + 1, & \text{otherwise,} \end{cases}$$

Let $T(z)$ be the ordinary generating function for t_n , i.e.,

$$T(z) = t_0 + t_1z + t_2z^2 + \dots + t_nz^n + \dots$$

Show that $T(z) = \frac{z^2 - z + 2}{(z-1)^2(z+2)}$.

Solution

$$zT(z) = t_0z + t_1z^2 + t_2z^3 + \dots + t_{n-1}z^n + \dots$$

$$z^2T(z) = t_0z^2 + t_1z^3 + t_2z^4 + \dots + t_{n-2}z^n + \dots$$

2 points for finding t_{n-2} and t_{n-1} generating functions

$$\text{Let } G(z) = 1 + z + z^2 + \dots + z^n + \dots$$

2 points for recognizing the need for $G(z)$

$$\frac{1}{2}zT(z) + \frac{1}{2}z^2T(z) + G(z) = 1 + (\frac{1}{2}t_0 + 1)z + (\frac{1}{2}t_1 + \frac{1}{2}t_0 + 1)z^2 + \dots + (\frac{1}{2}t_{n-1} + \frac{1}{2}t_{n-2} + 1)z^n$$

1 point for combining above parts correctly

If we subtract $\frac{1}{2}z$, this is $T(z)$.

$$T(z) = \frac{1}{2}zT(z) + \frac{1}{2}z^2T(z) + G(z) - \frac{1}{2}z$$

2 points for correctly fixing the previous step to make $T(z)$

Multiplying everything by 2,

$$2T(z) = zT(z) + z^2T(z) + 2G(z) - z$$

Moving $T(z)$ terms,

$$zT(z) + z^2T(z) - 2T(z) = z - 2G(z)$$

Combining $T(z)$ and substituting $G(z)$,

$$(z^2 + z - 2)T(z) = z - \frac{2}{1-z}$$

2 points for condensing terms and substituting $G(z)$

Combining terms/factoring,

$$(z+2)(z-1)T(z) = \frac{z(1-z)-2}{1-z}$$

Simplifying,

$$(z+2)(z-1)T(z) = \frac{-z^2+z-2}{1-z}$$

Dividing,

$$T(z) = \frac{-z^2+z-2}{(1-z)(z+2)(z-1)}$$

Multiplying top and bottom by -1 ($(1-z) \cdot -1 = (z-1)$),

$$T(z) = \frac{z^2 - z + 2}{(z-1)(z+2)(z-1)}$$

$$= \frac{z^2 - z + 2}{(z-1)^2(z+2)}$$

1 point for the -1 step needed to get the correct form.

As desired.

(c) [**10 Points**] We observe the following (you do not need to prove it):

$$\frac{z^2 - z + 2}{(z-1)^2(z+2)} = \frac{2}{3(1-z)^2} - \frac{1}{9(1-z)} + \frac{4}{9(1+\frac{z}{2})}.$$

Use the above and part (b) to show that

$$t_n = \frac{1}{9} \left(6n + 5 + 4 \left(-\frac{1}{2} \right)^n \right)$$

Solution

From the last part,

$$\begin{aligned} T(n) &= \frac{z^2 - z + 2}{(z-1)^2(z+2)} \\ &= \frac{2}{3(1-z)^2} - \frac{1}{9(1-z)} + \frac{4}{9(1+\frac{z}{2})} \\ \frac{2}{3(1-z)^2} &= \frac{2}{3} \frac{1}{(1-z)^2} = \frac{2}{3} \sum_{n=0}^{\infty} (n+1)z^n \end{aligned}$$

3 points for this fraction

$$\frac{1}{9(1-z)} = \frac{1}{9} \frac{1}{(1-z)} = \frac{1}{9} \sum_{n=0}^{\infty} z^n$$

3 points for this fraction

$$\frac{4}{9(1+\frac{z}{2})} = \frac{4}{9} \frac{1}{(1+\frac{z}{2})} = \frac{4}{9} \frac{1}{(1-\frac{-z}{2})} = \frac{4}{9} \sum_{n=0}^{\infty} \left(\frac{-z}{2} \right)^n$$

3 points for this fraction

So the n th term will be,

$$\begin{aligned} t_n &= \frac{2}{3}(n+1) - \frac{1}{9} + \frac{4}{9} \cdot \left(\frac{-1}{2} \right)^n \\ &= \frac{1}{9} (6(n+1) - 1 + (-\frac{1}{2})^n) \\ &= \frac{1}{9} (6n + 5 + 4(-\frac{1}{2})^n) \end{aligned}$$

As desired.

1 point for getting the n th term.

PARTITION(A, B, n)

Input: Two non-overlapping arrays A and B containing n numbers each, where n is a power of two.

Output: Rearrange the numbers in A and B such that no number in A is larger than any number in B .

1. **if** $n = 1$ **then**
2. **if** the number in A is larger than the one in B **then** swap the two numbers
3. **else**
4. let A_L (resp. B_L) denote the left half of A (resp. B) and let A_R (resp. B_R) denote its right half
5. PARTITION($A_L, B_L, \frac{n}{2}$)
6. PARTITION($A_R, B_R, \frac{n}{2}$)
7. PARTITION($A_L, B_R, \frac{n}{2}$)
8. PARTITION($A_R, B_L, \frac{n}{2}$)
9. **return**

REC-SELECTION-SORT(A, n)

Input: An array A containing n numbers, where n is a power of two.

Output: The numbers in A rearranged in nondecreasing order of value.

1. **if** $n > 1$ **then**
2. let A_L denote the left half of A and let A_R denote its right half
3. PARTITION($A_L, A_R, \frac{n}{2}$)
4. REC-SELECTION-SORT($A_L, \frac{n}{2}$)
5. REC-SELECTION-SORT($A_R, \frac{n}{2}$)
6. **return**

Figure 2: The recursive version of the selection sort algorithm.

Question 2. [25 Points] Parallel Recursive Selection Sort. When Pramod¹ was a student, he designed a recursive version of the selection sort algorithm which is more I/O-efficient. Figure 2 shows the high-level structure of the serial version of the algorithm. This question asks you to parallelize it and derive its parallel performance bounds.

¹Pramod Ganapathi – currently a faculty member of SBUCS.

- (a) [**10 Points**] Parallelize the PARTITION function. You can simply put the *spawn* and *sync* keywords at appropriate locations inside the function in Figure 2 to show how to parallelize it. Analyze its work, span, parallelism, and parallel running time (under a greedy scheduler).

Solution

A *spawn* before Line 5. **0.5 pt.**

A *sync* after Line 6. **0.5 pt.**

A *spawn* before Line 7. **0.5 pt.**

A *sync* after Line 8. **0.5 pt.**

Work: $T_1(n) = 4T_1(n/2) + \Theta(1) = \Theta(n^2)$. Base case $T_1(n) = \Theta(1)$ if $n \leq 1$. **2 pt (You lose 0.5 if no base case).**

Span: $T_\infty(n) = 2T_\infty(n/2) + \Theta(1) = \Theta(n)$. Base case $T_\infty(n) = \Theta(1)$ if $n \leq 1$. **2 pt (You lose 0.5 if no base case).**

Parallelism: $T(n) = T_1(n)/T_\infty(n) = \Theta(n)$. **2 pt.**

Parallel running time: $T_p(n) = \Theta(T_1(n)/p + T_\infty(n)) = \Theta(n^2/p + n)$. **2 pt.**

- (b) [**15 Points**] Parallelize the REC-SELECTION-SORT function. As in part (a), you can simply put the *spawn* and *sync* keywords at appropriate locations inside the function in Figure 2 to show how to parallelize it. Analyze its work, span, parallelism, and parallel running time (under a greedy scheduler).

Solution

A *spawn* before Line 4. **1.5 pt.**

A *sync* after Line 5. **1.5 pt.**

Work: $T_1(n) = 2T_1(n/2) + \Theta(n^2) = \Theta(n^2)$. Base case $T_1(n) = \Theta(1)$ if $n \leq 1$. **3 pt (You lose 0.5 if no base case).**

Span: $T_\infty(n) = T_\infty(n/2) + \Theta(n) = \Theta(n)$. Base case $T_\infty(n) = \Theta(1)$ if $n \leq 1$. **3 pt (You lose 0.5 if no base case).**

Parallelism: $T(n) = T_1(n)/T_\infty(n) = \Theta(n)$. **3 pt.**

Parallel running time: $T_p(n) = \Theta(T_1(n)/p + T_\infty(n)) = \Theta(n^2/p + n)$ **3 pt.**

RESIZE-LOCKER(L)

1. $m \leftarrow L.numItems$
2. **if** $m > 0$ **then**
3. **allocate** array *newslots* of size $2m$
4. copy the m items from $L.slots$ to the first m slots of *newslots* and set the remaining m slots to NIL
5. **else**
6. *newslots* $\leftarrow$ NIL
7. **endif**
8. **free** $L.slots$
9. $L.slots \leftarrow newslots$
10. $L.numSlots \leftarrow 2m$

LOCKER-STORE(L, x)

1. **if** $L.numSlots = 0$ **then**
2. **allocate** $L.slots$ with 2 slots
3. $L.numSlots \leftarrow 2, L.numItems \leftarrow 0$
4. $L.slots[1] \leftarrow \text{NIL}, L.slots[2] \leftarrow \text{NIL}$
5. **endif**
6. $f \leftarrow \text{FALSE}, n \leftarrow L.numSlots$
7. **while** $f = \text{FALSE}$ **do**
8. $k \leftarrow \text{RANDOM}(1, n)$
9. **if** $L.slots[k] = \text{NIL}$ **then**
10. $f \leftarrow \text{TRUE}$
11. $L.slots[k] \leftarrow x$
12. $L.numItems \leftarrow L.numItems + 1$
13. **endif**
14. **endwhile**
15. **if** $L.numItems \geq \frac{2}{3} \times L.numSlots$ **then**
16. RESIZE-LOCKER(L)
17. **endif**

LOCKER-RETRIEVE(L)

1. **if** $L.numItems = 0$ **then** $x \leftarrow \text{NIL}$
2. **else**
3. $f \leftarrow \text{FALSE}, n \leftarrow L.numSlots$
4. **while** $f = \text{FALSE}$ **do**
5. $k \leftarrow \text{RANDOM}(1, n)$
6. **if** $L.slots[k] \neq \text{NIL}$ **then**
7. $f \leftarrow \text{TRUE}$
8. $x \leftarrow L.slots[k]$
9. $L.slots[k] \leftarrow \text{NIL}$
10. $L.numItems \leftarrow L.numItems - 1$
11. **endif**
12. **endwhile**
13. **endif**
14. **if** $L.numItems \leq \frac{1}{3} \times L.numSlots$ **then**
15. RESIZE-LOCKER(L)
16. **endif**
17. **return** x

Figure 3: The Locker data structure.

Question 3. [25 Points] Store-Retrieve Lockers. Figure 3 shows the locker data structure L that maintains a resizable array $L.slots$ and supports the following two operations.

- LOCKER-STORE(L, x) stores an item x in a random empty slot of $L.slots$, and
- LOCKER-RETRIEVE(L) removes an item from a random nonempty slot of $L.slots$.

Each slot stores at most one item. The total number of slots in $L.slots$ is given by $L.numSlots$,

and the number of items currently stored in the data structure is given by $L.numItems$.

The $RESIZE-LOCKER(L)$ function resizes $L.slots$ as soon as one of the following two events occurs.

- $LOCKER-STORE(L, x)$ detects immediately after inserting x that

$$L.numItems \geq \frac{2}{3} \times L.numSlots \quad (\text{see Line 15})$$

- $LOCKER-RETRIEVE(L)$ detects immediately after removing an item that

$$L.numItems \leq \frac{1}{3} \times L.numSlots \quad (\text{see Line 14})$$

In both cases, $L.slots$ is resized to $L.numSlots = 2 \times L.numItems$. Observe that the smallest non-zero size $L.slots$ can have is 2 (see Lines 1–5 of $LOCKER-STORE$).

To insert an item into L , $LOCKER-STORE$ repeatedly chooses a slot in $L.slot$ uniformly at random until it finds an empty slot and stores the item in that slot (see Lines 6–14).

To retrieve an item from L , $LOCKER-RETRIEVE$ repeatedly chooses a slot in $L.slot$ uniformly at random until it finds a nonempty slot and removes the item from that slot (see Lines 3–12).

- (a) [**5 Points**] Show that the expected number of times the **while** loop in Lines 7–14 of $LOCKER-STORE$ needs to execute to find an empty spot in $L.slots$ is $\frac{n}{n-m}$, where $n = L.numSlots$ and $m = L.numItems$ at the time of execution. Also, show that the loop finds an empty spot in $\mathcal{O}(\log n)$ iterations w.h.p. in n .

Solution

Part1:

Let's define X_i at the i th iteration of the loop as follows-

$$X_i = \begin{cases} 1 & \text{if the selected slot is empty} \\ 0 & \text{otherwise} \end{cases}$$

Now probability of having an empty slot at the i -th iteration is,

$$P[X_i = 1] = \frac{n - m}{n}$$

Hence,

$$E[X_i] = P[X_i = 1] = \frac{n - m}{n}$$

1 point for finding $E[X_i]$

Let, k is the expected number of times the loop needs to execute to find an empty slot. Then,

$$E[X] = \sum_{i=1}^k E[X_i] = k \cdot \frac{n-m}{n}$$

Since, after k iterations, LOCKER-STORE finds a single empty slot hence,

$$k \cdot \frac{n-m}{n} = 1$$

$$k = \frac{n}{n-m}$$

Therefore, the expected number of time the while loop in lines 7-14 of LOCKER-STORE needs to execute to find an empty spot in L.slots is $\frac{n}{n-m}$

1 points for finding number of iterations

Part2:

We know from part1,

$$P[X_i = 1] = \frac{n-m}{n}$$

Therefore,

$$P[X_i = 0] = 1 - \frac{n-m}{n}$$

Given that the loop finds an empty spot in $O(\log n)$ iterations. Hence,

$$P[X_i = 0 \text{ in } \log n \text{ iterations}] = \left(1 - \frac{n-m}{n}\right)^{\log n} = \left(\frac{m}{n}\right)^{\log n}$$

1 point for finding above probability

Therefore,

$$P[\text{at least one } X_i = 1 \text{ in } \log n \text{ iterations}] = 1 - \left(\frac{m}{n}\right)^{\log n}$$

1 point for finding above probability

We can see, $\frac{n}{3} < m < \frac{2n}{3}$

Therefore,

$$P[\text{at least one } X_i = 1 \text{ in } \log n \text{ iterations}] > 1 - \left(\frac{\frac{2n}{3}}{\frac{n}{3}}\right)^{\log n} = 1 - \left(\frac{2}{3}\right)^{\log n} = 1 - \left(\frac{3}{2}\right)^{-\log n} = 1 - \left(\frac{3}{2}\right)^{\log(\frac{1}{n})}$$

$$= 1 - \frac{1}{n^{\log(\frac{3}{2})}}$$

This ends the proof that the loop finds a nonempty spot in $O(\log n)$ iterations w.h.p in n.

1 point for this proof

- (b) [5 Points] Show that the expected number of times the **while** loop in Lines 4–12 of LOCKER-RETRIEVE needs to execute to find a nonempty spot in $L.slots$ is $\frac{n}{m}$, where $n = L.numSlots$ and $m = L.numItems$ at the time of execution. Also, show that the loop finds a nonempty spot in $\mathcal{O}(\log n)$ iterations w.h.p. in n .

Solution

Part1:

Let's define X_i at the i th iteration of the loop as follows-

$$X_i = \begin{cases} 1 & \text{if the selected slot is nonempty} \\ 0 & \text{otherwise} \end{cases}$$

Now probability of having an nonempty slot at the i -th iteration is,

$$P[X_i = 1] = \frac{m}{n}$$

Hence,

$$E[X_i] = P[X_i = 1] = \frac{m}{n}$$

1 point for finding $E[X_i]$

Let, k is the expected number of times the loop needs to execute to find a nonempty slot. Then,

$$E[X] = \sum_{i=1}^k E[X_i] = k \cdot \frac{m}{n}$$

Since, after k iterations, LOCKER-RETRIEVE finds a single nonempty slot,

$$k \cdot \frac{m}{n} = 1$$

$$k = \frac{n}{m}$$

Therefore, the expected number of times the while loop in lines 4-12 of LOCKER-RETRIEVE needs to execute to find a nonempty spot in $L.slots$ is $\frac{n}{m}$

1 points for this proof

Part2:

We know from part1,

$$P[X_i = 1] = \frac{m}{n}$$

Therefore,

$$P[X_i = 0] = 1 - \frac{m}{n}$$

Given that the loop finds a nonempty spot in $O(\log n)$ iterations. Hence,

$$P[X_i = 0 \text{ in } \log n \text{ iterations}] = \left(1 - \frac{m}{n}\right)^{\log n}$$

1 point for finding above probability

Therefore,

$$P[\text{at least one } X_i = 1 \text{ in } \log n \text{ iterations}] = 1 - \left(\frac{n-m}{n}\right)^{\log n}$$

1 point for finding above probability

Again we know, $\frac{n}{3} < m < \frac{2n}{3}$

Therefore,

$$\begin{aligned} P[\text{at least one } X_i = 1 \text{ in } \log n \text{ iterations}] &> 1 - \left(\frac{n - \frac{n}{3}}{n}\right)^{\log n} = 1 - \left(\frac{2}{3}\right)^{\log n} = 1 - \left(\frac{3}{2}\right)^{-\log n} = 1 - \left(\frac{3}{2}\right)^{\log(\frac{1}{n})} \\ &= 1 - \frac{1}{n^{\log(\frac{3}{2})}} \end{aligned}$$

This ends the proof that the loop finds an empty spot in $O(\log n)$ iterations w.h.p in n .

1 points for this proof

- (c) [**3 Points**] In order to find the amortized costs of the operations performed on L we will use the following potential function:

$$\Phi(L_i) = c \times |2 \times L.numItems - L.numSlots|,$$

where, L_i is the state of L after the i -th ($i \geq 0$) operation is performed on it assuming that L was initially empty, and c is a positive constant.

Argue that this potential function guarantees that the total amortized cost will always be an upper bound on the total actual cost.

Solution

To prove that the total amortized cost will always be an upper bound on the total actual cost, we have to show that,

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

One way of achieving that is to find a ϕ such that, $\phi(L_0) = 0$ and $\phi(L_n) \geq 0$

Now, when the structure is empty, $L.numItems = 0$ and $L.numSlots = 0$

Hence, $\phi(L_0) = c \times |2 \times 0 - 0| = 0$

$|2 \times L.numItems - L.numSlots| \geq 0$ since it's the absolute value of the difference. Also since, $c > 0$ hence, $\phi(L_n) \geq 0$. As we have proved that, $\phi(L_0) = 0$ and $\phi(L_n) \geq 0$ hence,

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

This ends the proof that the given potential function guarantees that the total amortized cost will always be an upper bound on the total actual cost.

1.5 point for finding $\phi(L_0)$

1.5 point for finding $\phi(L_n)$

(d) [**12 Points**] Use the potential function from part (c) and your results from parts (a) and (b) to show that the amortized costs of

- RESIZE-LOCKER is 0,
- LOCKER-STORE is $\mathcal{O}(\log n)$ w.h.p. in n , and
- LOCKER-RETRIEVE is $\mathcal{O}(\log n)$ w.h.p. in n ,

where, $n = L.numSlots$ at the time of execution.

Solution

We know, that the amortized cost of the i th operation can be defined as,

$$\hat{c}_i = \text{actual cost} + \text{potential change due to that operation} = c_i + \phi(L_i) - \phi(L_{i-1})$$

RESIZE-LOCKER:

$$c_i = 2m \text{ (copy } m \text{ items + setting } m \text{ items to NIL)}$$

$$\phi(L_{i-1}) = c|2m - n|$$

$$\phi(L_i) = c|2m - 2m| = 0$$

$$\text{Hence, } \hat{c}_i = 2m + (0 - c|2m - n|) = 2m - c|2m - n|$$

Now, if, $m = \frac{n}{3}$ then,

$$\hat{c}_i = \frac{2n}{3} + (0 - c|\frac{2n}{3} - n|) = \frac{2n}{3} - \frac{cn}{3}$$

If, $c \geq 2$ then, $\hat{c}_i = 0$

Again, if $m = \frac{2n}{3}$ then,

$$\hat{c}_i = \frac{4n}{3} - \frac{cn}{3}$$

if $c \geq 4$ then $\hat{c}_i = 0$

that means the amortized cost of RESIZE-LOCKER is 0

1.5 points for finding $c_i, \phi(L_{i-1}), \phi(L_i)$

2.5 points for the proof

LOCKER-STORE:

$$c_i = \log n + 0 \text{ (cost of while loop + resize)} = \log n$$

$$\phi(L_{i-1}) = c|2m - n|$$

$$\phi(L_i) = c|2(m+1) - n| \text{ (as 1 item added at each step)}$$

Now,

$$\hat{c}_i = c_i + \phi(L_i) - \phi(L_{i-1}) = \log n + c|2m + 2 - n| - c|2m - n|$$

if $2m - n > 0$, then $\hat{c}_i = \log n + 2c = O(\log n)$

else if, $2m - n < 0$, then $\hat{c}_i = \log n - 2c = O(\log n)$

That shows the amortized cost of LOCKER-STORE is $O(\log n)$

1.5 points for finding $c_i, \phi(L_{i-1}), \phi(L_i)$

2.5 points for the proof

LOCKER-RETRIEVE:

$$c_i = \log n + 0 = \log n$$

$$\phi(L_{i-1}) = c|2m - n|$$

$$\phi(L_i) = c|2m - 2 - n|$$

$$\hat{c}_i = \log n \pm \text{constant} = O(\log n)$$

This ends the proof that the amortized cost of the LOCKER-RETRIEVE is $O(\log n)$

1.5 points for finding $c_i, \phi(L_{i-1}), \phi(L_i)$

2.5 points for the proof